

**Java Backend Interview Series**  
**Sub-Chapter 1 10**  
**Java 9 to 21 Features**  
Modern Java In-Depth Tutorial

*Motivation + Before/After Code + Full Import Statements + 52 Q&A + Exercises*

# 1 Java 9 — Module System (JPMS) & JShell

## 1.1 Introduction

Before Java 9, all JARs lived on a flat classpath where any class could reference any other class — no encapsulation boundary existed above the package level. This caused "JAR hell": version conflicts, accidental use of internal JDK APIs (`sun.misc.Unsafe`), and runtime class-not-found failures. Java 9 introduced the Java Platform Module System (JPMS / Project Jigsaw) — a new unit above packages called the module.

## 1.2 The Problem — Classpath Hell

```
# Before Java 9 – flat classpath, no isolation

java -cp lib/app.jar:lib/guava-28.jar:lib/guava-30.jar MyApp

# Which Guava does each library see? Undefined.

# Internal APIs freely accessible:

sun.misc.Unsafe unsafe = sun.misc.Unsafe.getUnsafe(); // compiles!

// Breaks silently when JDK is upgraded.
```

*Before Java 9: flat classpath, zero enforcement*

## 1.3 The Solution — module-info.java

```
// module-info.java (no imports – special file, no import statements)

module com.myapp.service {

    requires java.sql; // dependency

    requires transitive com.myapp.model; // transitive dependency

    exports com.myapp.service.api; // visible to all modules

    exports com.myapp.service.impl to spring.core; // visible only to spring

    opens com.myapp.service.impl to spring.core; // allow deep reflection

    uses com.myapp.service.api.Plugin; // SPI consumer

    provides com.myapp.service.api.Plugin

    with com.myapp.service.impl.DefaultPlugin; // SPI provider

}
```

*module-info.java — complete syntax (no imports needed)*

| Directive  | Effect                 | Compile-time? | Runtime? |
|------------|------------------------|---------------|----------|
| requires X | Dependency on module X | Yes           | Yes      |

| Directive              | Effect                            | Compile-time? | Runtime? |
|------------------------|-----------------------------------|---------------|----------|
| requires transitive X  | Implies X to downstream consumers | Yes           | Yes      |
| exports pkg            | pkg visible to ALL modules        | Yes           | Yes      |
| exports pkg to M       | pkg visible only to M             | Yes           | Yes      |
| opens pkg to M         | Deep reflection from M allowed    | No            | Yes      |
| uses SPI               | ServiceLoader consumer            | Yes           | Yes      |
| provides SPI with Impl | ServiceLoader provider            | Yes           | Yes      |

## 1.4 exports vs opens

```
// exports: compile-time + runtime usage, NO deep reflection

// opens: runtime deep reflection ONLY

exports com.myapp.api;

// Allowed: import com.myapp.api.Foo; Foo f = new Foo();

// Blocked: Field f = Foo.class.getDeclaredField("secret");
// f.setAccessible(true); // -> InaccessibleObjectException

opens com.myapp.impl to spring.core;

// Allowed: Spring injects @Autowired private UserService field

// Blocked: import com.myapp.impl.InternalClass; // compile error
```

*exports vs opens*

## 1.5 Collection Factories (Java 9)

```
import java.util.List;
import java.util.Set;
import java.util.Map;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;

// BEFORE Java 9

List<String> names = new ArrayList<>(Arrays.asList("Alice","Bob","Carol"));

Map<String,Integer> scores = new HashMap<>();

scores.put("Alice",95); scores.put("Bob",87);

// AFTER Java 9 – concise, truly immutable
```

```
List<String> names = List.of("Alice","Bob","Carol");

Set<Integer> ids = Set.of(1,2,3);

Map<String,Integer> scores = Map.of("Alice",95,"Bob",87);

// More than 10 map entries:

Map<String,Integer> big = Map.ofEntries(
    Map.entry("Alice",95), Map.entry("Bob",87), Map.entry("Carol",92)
);

// Gotchas:

List.of("a",null,"b"); // NullPointerException – null-hostile

Set.of(1,1,2); // IllegalArgumentException – duplicate
```

*Java 9 collection factories*

**WARNING:** `List.of()`, `Set.of()`, `Map.of()` throw `UnsupportedOperationException` on mutation and `NullPointerException` for null elements.

## 1.6 Stream Enhancements (Java 9)

```
import java.util.Optional;
import java.util.List;
import java.util.stream.Stream;
import java.util.stream.Collectors;

// takeWhile – stops at first non-matching element

Stream.of(2,4,6,5,8)

    .takeWhile(n -> n % 2 == 0)

    .forEach(System.out::print); // 2 4 6 (stops at 5)

// dropWhile – drops while predicate is true

Stream.of(2,4,6,5,8)

    .dropWhile(n -> n % 2 == 0)

    .forEach(System.out::print); // 5 8

// iterate with stop condition (finite stream)

Stream.iterate(1, n -> n < 100, n -> n*2) // 1,2,4,8,16,32,64

    .forEach(System.out::print);

// Optional.stream() – flat-map over a stream of Optionals

List<Optional<String>> opts =

    List.of(Optional.of("a"), Optional.empty(), Optional.of("c"));

List<String> present =
```

```
opts.stream().flatMap(Optional::stream).toList(); // [a,c]
```

*Java 9 Stream additions*

TIP: `takeWhile` is NOT the same as `filter`. `filter` tests every element; `takeWhile` stops at the first non-matching element. For sorted streams, `takeWhile` is usually what you want.

## 1.7 JShell — Interactive REPL

```
$ jshell

jshell> int x = 42

x ==> 42

jshell> String greet(String name) { return "Hello, " + name; }

| created method greet(String)

jshell> greet("World")

$3 ==> "Hello, World"

jshell> List.of(1,2,3).stream().map(n -> n*n).toList()

$4 ==> [1, 4, 9]

jshell> /exit
```

*JShell session (shell commands, no imports needed)*

## 1.8 Summary

JPMS replaces the flat classpath with a module graph. Modules declare dependencies (requires), visibility (exports), and reflective access (opens). This eliminates version ambiguity, prevents access to internal APIs, and enables modular JDK images. Collection factories give concise immutable collections. `Stream.takeWhile/dropWhile` enable ordered stream processing. JShell provides instant experimentation.

## 2 Java 10–11 — var, String Methods & HTTP Client

### 2.1 Introduction

Java 10 introduced local-variable type inference via the `var` keyword — the compiler deduces the type from the right-hand side at compile time. This is NOT dynamic typing. Java 11 standardised the `HttpClient` API, added essential `String` utility methods, and extended `var` to lambda parameters with annotations.

### 2.2 var — Local-Variable Type Inference

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

// BEFORE – repetitive type declarations

ArrayList<String> list = new ArrayList<String>();
HashMap<String,List<Integer>> map = new HashMap<String,List<Integer>>();

// AFTER – var infers at compile time; same bytecode

var list = new ArrayList<String>();
var map = new HashMap<String,List<Integer>>();

// for-each
for (var entry : map.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}

// Lambda parameters with annotation (Java 11)
list.stream()
    .filter((@NonNull var s) -> !s.isBlank())
    .forEach(System.out::println);

// Does NOT compile:
var x; // no initialiser – cannot infer type
var y = null; // null has no type to infer
```

*var: before and after*

| var works here                  | var does NOT work here        |
|---------------------------------|-------------------------------|
| Local variable with initialiser | Method/constructor parameters |
| for-each loop variable          | Method return types           |
| Traditional for-loop variable   | Class or instance fields      |

| var works here                         | var does NOT work here       |
|----------------------------------------|------------------------------|
| Try-with-resources variable            | var x; (no initialiser)      |
| Lambda param with annotation (Java 11) | var y = null; (type unknown) |

**WARNING:** `var list = new ArrayList<>()` infers `ArrayList` (not `List`). Prefer an explicit type when you want to code to an interface. `var` is a reserved type name, not a keyword.

## 2.3 Java 11 String Methods

```
// strip() – Unicode-aware; trim() only strips ASCII
" hello ".strip(); // "hello"
" hello ".stripLeading(); // "hello "
" hello ".stripTrailing(); // " hello"

// isBlank() – true if empty or only whitespace
"".isBlank(); // true
" ".isBlank(); // true
"a".isBlank(); // false

// lines() – Stream<String> split on \n, \r, \r\n
"line1\nline2\nline3".lines().count(); // 3

// repeat(n)
"ab".repeat(3); // "ababab"
"=".repeat(40); // "====================================="

// Java 12 additions
"hello".indent(4); // " hello\n"
" 42 ".strip().transform(Integer::parseInt); // Integer 42
```

*Java 11 String methods (java.lang.String — no imports needed)*

## 2.4 Java 11 HTTP Client

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;
import java.util.concurrent.CompletableFuture;
```

```
// Synchronous GET

HttpClient client = HttpClient.newBuilder()
    .version(HttpClient.Version.HTTP_2)
    .connectTimeout(Duration.ofSeconds(10))
    .followRedirects(HttpClient.Redirect.NORMAL)
    .build();

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/users/1"))
    .header("Accept", "application/json")
    .GET().build();

HttpResponse<String> response =
    client.send(request, HttpResponse.BodyHandlers.ofString());

System.out.println(response.statusCode()); // 200
System.out.println(response.body()); // {"id":1,...}

// Asynchronous POST

CompletableFuture<HttpResponse<String>> future = client.sendAsync(
    HttpRequest.newBuilder()
        .uri(URI.create("https://api.example.com/users"))
        .header("Content-Type", "application/json")
        .POST(HttpRequest.BodyPublishers.ofString("""{"name":"Alice"}"""))
        .build(),
    HttpResponse.BodyHandlers.ofString()
);

future.thenAccept(r -> System.out.println("Status: " + r.statusCode()));
```

*Java 11 HTTP Client — sync and async*

**TIP:** `HttpClient` is thread-safe. Create one per application, not one per request. It manages HTTP/2 connection pooling automatically.

## 2.5 Summary

`var` eliminates repetitive type declarations with full compile-time type safety. Use it where the RHS makes the type obvious. Java 11 String methods fill long-standing gaps: `strip()` is Unicode-aware (unlike `trim()`); `lines()` streams lines; `isBlank()` tests empty/whitespace. The `HttpClient` is the correct modern API — `URLConnection` is legacy.



## 3 Java 14–16 — Records

### 3.1 Introduction

A record is a concise syntax for immutable data-carrier classes. Before records, a correct two-field value type required 35+ lines. Records compress everything to a single line while generating the canonical constructor, accessors, equals, hashCode, and toString automatically.

### 3.2 Before Records — Boilerplate Problem

```
import java.util.Objects;

// BEFORE Java 16 – 35+ lines for two fields

public final class Point {

    private final double x;

    private final double y;

    public Point(double x, double y) { this.x=x; this.y=y; }

    public double x() { return x; }

    public double y() { return y; }

    @Override public boolean equals(Object o) {

        if (!(o instanceof Point other)) return false;

        return Double.compare(x,other.x)==0 && Double.compare(y,other.y)==0;

    }

    @Override public int hashCode() { return Objects.hash(x,y); }

    @Override public String toString() { return "Point[x="+x+",y="+y+"]"; }

}
```

*Before records: 35 lines for 2 fields*

### 3.3 After Records — One Line

```
// AFTER Java 16 – one line replaces everything above

record Point(double x, double y) {}

// Compiler auto-generates:

// private final double x, y;

// Point(double x, double y) – canonical constructor

// double x() { return x; } – accessor (NOT getX!)

// double y() { return y; }

// equals(Object) – compares all components
```

```
// hashCode() – based on all components

// toString() – "Point[x=1.0, y=2.0]"

Point p1 = new Point(1.0, 2.0);

Point p2 = new Point(1.0, 2.0);

p1.equals(p2); // true

p1.x(); // 1.0 (accessor – NOT getX())

p1.toString(); // "Point[x=1.0, y=2.0]"
```

*After records: 1 line*

### 3.4 Compact Canonical Constructor — Validation

```
// No parameter list, no this.field = field

// Compiler inserts assignments after your body

record Temperature(double celsius) {

    Temperature {

        if (celsius < -273.15)

            throw new IllegalArgumentException("Below absolute zero: "+celsius);

        celsius = Math.round(celsius * 100.0) / 100.0; // normalise 2dp

        // this.celsius = celsius <-- added by compiler automatically

    }

}

record Range(int min, int max) {

    Range {

        if (min > max) throw new IllegalArgumentException(

            "min "+min+" > max "+max);

    }

    int span() { return max - min; }

    boolean contains(int v) { return v >= min && v <= max; }

}

new Temperature(-300); // throws IllegalArgumentException immediately
```

*Compact canonical constructor*

### 3.5 Records with Interfaces and Static Members

```
interface Drawable { double area(); }

record Circle(double radius) implements Drawable {

    static final Circle UNIT = new Circle(1.0); // static field OK

    static Circle of(double r) { return new Circle(r); } // factory

    @Override

    public double area() { return Math.PI * radius * radius; }

    double circumference() { return 2 * Math.PI * radius; }

    // Override generated accessor

    public double radius() { return Math.abs(this.radius); }

}
```

*Records implementing interfaces with custom methods*

### 3.6 Records as Spring Boot DTOs

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;

// Request DTO

record CreateProductRequest(

    @NotBlank String name,

    @Positive BigDecimal price,

    @NotBlank String category

) {}

// Response DTO with normalisation

record ProductResponse(Long id, String name, BigDecimal price) {

    ProductResponse {

        price = price.setScale(2, RoundingMode.HALF_UP);

    }

}
```

```
}

@PostMapping("/products")

public ResponseEntity<ProductResponse> create(
    @Valid @RequestBody CreateProductRequest req) {

    Product p = productService.create(req.name(), req.price(), req.category());

    return ResponseEntity.ok(
        new ProductResponse(p.getId(), p.getName(), p.getPrice()));

}
```

*Records as Spring Boot DTOs*

**WARNING:** Records cannot extend other classes (they implicitly extend `Record`). They are implicitly final. Instance fields are always private final — you cannot add mutable instance fields. Static fields and methods are allowed.

### 3.7 Summary

Records eliminate immutable data-class boilerplate. The compiler generates the canonical constructor, accessors (`x()` not `getX()`), equals, hashCode, and toString. Compact constructors add validation without repeating assignments. Records implement interfaces, have static members, and override accessors. Ideal for DTOs, value objects, and CQRS command/query types.

## 4 Java 14–17 — Pattern Matching & Sealed Classes

### 4.1 Introduction

Pattern matching for `instanceof` (Java 16) and sealed classes (Java 17) together enable exhaustive type-based dispatch. They eliminate the "else-if chain of `instanceof`" anti-pattern and give the compiler enough information to detect missing cases at compile time.

### 4.2 Pattern Matching for `instanceof`

```
// BEFORE Java 16 – redundant cast after instanceof

if (obj instanceof String) {

    String s = (String) obj; // cast already known safe by compiler

    System.out.println(s.toUpperCase());

} else if (obj instanceof Integer) {

    Integer i = (Integer) obj; // another redundant cast

    System.out.println(i * 2);

}


// AFTER Java 16 – binding variable declared in pattern

if (obj instanceof String s) {

    System.out.println(s.toUpperCase()); // s bound, no cast

} else if (obj instanceof Integer i) {

    System.out.println(i * 2);

}


// Combined guard in same expression

if (obj instanceof String s && s.length() > 5) {

    System.out.println("Long string: " + s);

}


// Negation – s is in scope after early return

if (!(obj instanceof String s)) return;

System.out.println(s.toUpperCase()); // definitely a String here
```

*Pattern matching instanceof: before and after*

## 4.3 Sealed Classes — Controlled Inheritance

```
// Sealed interface – only these three types can implement it

public sealed interface Shape permits Circle, Rectangle, Triangle {}

public record Circle(double radius) implements Shape {}

public record Rectangle(double w, double h) implements Shape {}

public final class Triangle implements Shape {

    private final double base, height;

    public Triangle(double b, double h) { this.base=b; this.height=h; }

    public double base() { return base; }

    public double height() { return height; }

}

// Exhaustive switch – no default needed

// Adding a new Shape causes this switch to fail to compile – by design

static double area(Shape shape) {

    return switch (shape) {

        case Circle c -> Math.PI * c.radius() * c.radius();

        case Rectangle r -> r.w() * r.h();

        case Triangle t -> 0.5 * t.base() * t.height();

    };

}
```

*Sealed interface with exhaustive pattern switch*

**TIP:** Sealed + records + pattern switch is the Java equivalent of algebraic data types (Scala case classes, Rust enums). Ideal for domain events, command/query types, or any finite variant set.

## 4.4 Switch Expressions (Java 14)

```
// BEFORE – traditional switch (fall-through risk, no return value)

String label;

switch (day) {

    case MONDAY: case TUESDAY: label = "Start"; break;

    case FRIDAY: label = "End"; break;

    default: label = "Middle";

}
```

```
}

// AFTER Java 14 – expression with arrow syntax, no fall-through

String label = switch (day) {

    case MONDAY, TUESDAY -> "Start";

    case FRIDAY -> "End";

    default -> "Middle";

};

// Block case with yield

int fee = switch (tier) {

    case BRONZE -> 0;

    case SILVER -> 5;

    case GOLD -> {

        int base = computeBase();

        yield base > 100 ? base - 10 : base; // yield, not return

    }

};
```

*Switch expression: before and after*

## 4.5 Guarded Patterns with when (Java 21)

```
static String classify(Object obj) {

    return switch (obj) {

        case Integer i when i < 0 -> "Negative: " + i;

        case Integer i when i == 0 -> "Zero";

        case Integer i -> "Positive: " + i;

        case String s when s.isEmpty() -> "(empty)";

        case String s -> "String of length " + s.length();

        case null -> "null";

        default -> "Other: " + obj;

    };

}
```

```
classify(-5); // "Negative: -5"

classify(0); // "Zero"

classify("hi"); // "String of length 2"

classify(null); // "null"
```

*Guarded patterns in switch (Java 21)*

**WARNING:** Guard dominance: more specific patterns must appear before general ones. "case Integer i when i < 0" must precede "case Integer i" or a compile error occurs.

## 4.6 Summary

Pattern matching instanceof eliminates redundant casts. Sealed classes restrict type hierarchies to a declared set. Together they enable exhaustive pattern switches where the compiler enforces every subtype is handled. Switch expressions return values and eliminate fall-through. Guarded patterns (when) add inline conditions.



## 5 Java 15 — Text Blocks

### 5.1 Introduction

Before text blocks, multi-line strings (SQL, JSON, HTML, XML) required escape sequences and concatenation — hard to read, easy to break. Text blocks (finalised Java 15) introduce a literal that looks like the string it contains, with indentation managed at compile time.

### 5.2 Before vs After

```
// BEFORE Java 15

String sql = "SELECT u.id, u.name\n" +
"FROM users u\n" +
"WHERE u.active = true";

String json = "{\n  \"name\": \"Alice\",\n  \"age\": 30\n}";

// AFTER Java 15

String sql = """
SELECT u.id, u.name
FROM users u
WHERE u.active = true
""";

String json = """
{
"name": "Alice",
"age": 30
}
""";

// Interpolation with formatted() (no String.format needed)

String greeting = """
Dear %, your order #d has shipped. Total: $.2f
""".formatted("Alice", 12345, 99.95);
```

*Text blocks: before and after (java.lang.String — no imports needed)*

## 5.3 Indentation Stripping Rules

```
// Closing "" at same indent as content – 8 spaces stripped

String a = ""

SELECT *

FROM users

""; // -> "SELECT *\nFROM users\n"

// Closing "" at column 0 – NOTHING stripped (BUG!)

String b = ""

SELECT *

FROM users

""; // -> " SELECT *\n FROM users\n"

// stripIndent() – strips programmatically at runtime

String c = " line1\n line2\n".stripIndent(); // "line1\nline2\n"

// \s – preserve trailing space

// \ – line continuation (no newline)

String oneLine = ""

This is very long \

but on one line.

""; // "This is very long but on one line.\n"
```

*Text block indentation rules*

**WARNING:** Placing the closing "" at column 0 retains ALL leading whitespace on every line — almost always a bug. Align the closing delimiter with the content lines.

## 5.4 Summary

Text blocks eliminate escape sequences in multi-line strings. The opening triple-quote must be on its own line. Indentation is stripped at compile time based on closing delimiter position. Use `formatted()` for interpolation, `\s` for intentional trailing spaces, and `\` for line continuation. At runtime, a text block is a `String` constant — zero performance difference.

## 6 Java 21 — Virtual Threads & Structured Concurrency

### 6.1 Introduction

Java's traditional platform threads map 1:1 to OS threads (~1 MB stack each). For I/O-intensive services, the constraint is thread count, not CPU. Virtual threads (Project Loom, Java 21) let the JVM manage millions of lightweight threads that unmount from their carrier platform thread during blocking I/O — freeing the carrier for other tasks. No reactive programming required.

### 6.2 Creating Virtual Threads

```
import java.time.Duration;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.stream.IntStream;

// Single virtual thread

Thread vt = Thread.ofVirtual().name("worker").start(() -> {
    Thread.sleep(Duration.ofSeconds(1)); // yields carrier – NOT blocked!
    System.out.println("Done: " + Thread.currentThread());
});

// Virtual-thread-per-task executor (idiomatic)

try (ExecutorService exec = Executors.newVirtualThreadPerTaskExecutor()) {
    // 100,000 tasks, each on its own virtual thread
    IntStream.range(0, 100_000).forEach(i ->
        exec.submit(() -> {
            Thread.sleep(Duration.ofSeconds(1)); // yields carrier
            return fetchUser(i);
        })
    );
} // waits for all tasks

// Spring Boot 3.2+ – enable virtual threads in application.yml:
// spring.threads.virtual.enabled: true
```

*Virtual thread creation and executor usage*

| Aspect         | Platform Thread            | Virtual Thread            |
|----------------|----------------------------|---------------------------|
| Backed by      | OS thread (1:1)            | JVM continuation (M:N)    |
| Stack size     | ~1 MB fixed                | Few KB, grows dynamically |
| Max concurrent | Thousands (memory limited) | Millions                  |

| Aspect         | Platform Thread  | Virtual Thread             |
|----------------|------------------|----------------------------|
| Thread.sleep() | Blocks OS thread | Yields carrier — efficient |
| synchronized   | Works fine       | PINS carrier — avoid!      |
| I/O blocking   | Blocks OS thread | Unmounts carrier           |

**WARNING:** `synchronized` blocks PIN the virtual thread to its carrier, eliminating scalability. Replace `synchronized` with `ReentrantLock` (`java.util.concurrent.locks`) which allow unmounting. Diagnose: `-Djdk.tracePinnedThreads=full`

## 6.3 Structured Concurrency

```
import java.util.concurrent.StructuredTaskScope;
import java.util.concurrent.StructuredTaskScope.Subtask;

// ShutdownOnFailure: cancel all if any task fails
record UserAccount(User user, Account account) {}

UserAccount loadUser(long id) throws Exception {
    try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
        Subtask<User> ut = scope.fork(() -> userService.findById(id));
        Subtask<Account> at = scope.fork(() -> accountService.findById(id));
        scope.join().throwIfFailed();
        return new UserAccount(ut.get(), at.get());
    }
}

// ShutdownOnSuccess: first result wins (racing)
String fastCache(String key) throws Exception {
    try (var scope = new StructuredTaskScope.ShutdownOnSuccess<String>()) {
        scope.fork(() -> primaryCache.get(key));
        scope.fork(() -> secondaryCache.get(key));
        scope.join();
        return scope.result();
    }
}
```

*StructuredTaskScope: ShutdownOnFailure and ShutdownOnSuccess*

## 6.4 Scoped Values

```
import java.lang.ScopedValue;

// Immutable ThreadLocal alternative
static final ScopedValue<String> CURRENT_USER = ScopedValue.newInstance();
```

```
ScopedValue.where(CURRENT_USER, "alice").run(() -> {
    processRequest(); // CURRENT_USER.get() == "alice"
    // Child virtual threads automatically inherit this value
});

// After scope: value is gone – no cleanup needed

void processRequest() {
    String user = CURRENT_USER.get(); // "alice"
}
```

*Scoped Values — immutable context propagation*

## 6.5 Summary

Virtual threads write simple blocking code that scales to millions of concurrent tasks. The JVM unmounts a virtual thread during blocking, freeing the carrier for other work. Replace `synchronized` with `ReentrantLock` to avoid pinning. Spring Boot 3.2 enables virtual threads with one config line. Structured Concurrency ties subtask lifecycles to a scope. Scoped Values replace `ThreadLocal` with an immutable, auto-inherited alternative.

## 7 Java 21 — Sequenced Collections

### 7.1 Introduction

Before Java 21, first/last element access was inconsistent across collection types: `list.get(0)`, `deque.peekFirst()`, `sortedSet.first()`, `linkedHashMap.entrySet().iterator().next()`. Java 21 unified these with three new interfaces retrofitted onto existing types.

### 7.2 The New Interfaces

```
// SequencedCollection<E> – List, Deque, LinkedHashSet implement this

// (java.util package – no extra imports for standard types)

interface SequencedCollection<E> {

    E getFirst(); E getLast();

    void addFirst(E e); void addLast(E e);

    E removeFirst(); E removeLast();

    SequencedCollection<E> reversed(); // lazy view

}

// SequencedMap<K,V> – LinkedHashMap, TreeMap implement this

interface SequencedMap<K,V> {

    Map.Entry<K,V> firstEntry(); Map.Entry<K,V> lastEntry();

    Map.Entry<K,V> pollFirstEntry(); Map.Entry<K,V> pollLastEntry();

    SequencedMap<K,V> reversed();

}
```

*Sequenced interfaces (java.util — no extra imports needed)*

### 7.3 Before vs After

```
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;

// ArrayList

var list = new ArrayList<>(List.of("a", "b", "c"));

// BEFORE Java 21

list.get(0); // first – OK

list.get(list.size()-1); // last – fragile!
```

```
// AFTER Java 21

list.getFirst(); // "a"
list.getLast(); // "c"
list.addFirst("z"); // ["z","a","b","c"]
list.reversed(); // lazy reversed view

// LinkedHashMap
var map = new LinkedHashMap<String,Integer>();
map.put("one",1); map.put("two",2); map.put("three",3);

// BEFORE Java 21
map.entrySet().iterator().next(); // first entry – verbose!

// AFTER Java 21
map.firstEntry(); // Map.Entry<"one",1>
map.lastEntry(); // Map.Entry<"three",3>
map.reversed().forEach((k,v) -> System.out.println(k+": "+v));
```

*Sequenced collections: before and after*

## 7.4 Summary

SequencedCollection, SequencedSet, and SequencedMap give uniform first/last access to Java's ordered collection types. No migration needed: ArrayList, ArrayDeque, LinkedHashSet, and LinkedHashMap already implement these interfaces in Java 21. The reversed() method returns a lazy view without copying.

## 8 GraalVM, Project Panama & Future Directions

### 8.1 GraalVM Native Image

GraalVM compiles Java AOT to a standalone native binary. Startup ~10ms (vs ~2s JVM), RSS in tens of MB. Tradeoff: no dynamic class loading; reflection/proxies need config files.

```
# Build (requires GraalVM with native-image)

native-image -jar myapp.jar -o myapp

./myapp # ~15ms startup


# Generate reflection config via tracing agent

java -agentlib:native-image-agent=config-output-dir=META-INF/native-image \
-jar myapp.jar

# Creates: reflect-config.json, proxy-config.json, serialization-config.json


# Spring Boot 3+ native compilation

./mvnw -Pnative native:compile

./target/myapp # Spring context < 100ms
```

*GraalVM Native Image (shell commands — no Java imports)*

### 8.2 Foreign Function & Memory API (Project Panama)

```
import java.lang.foreign.Arena;
import java.lang.foreign.FunctionDescriptor;
import java.lang.foreign.Linker;
import java.lang.foreign.MemorySegment;
import java.lang.foreign.ValueLayout;
import java.lang.invoke.MethodHandle;

// Call C strlen() from Java — no C code, no JNI headers

try (Arena arena = Arena.ofConfined()) {

    MemorySegment str = arena.allocateFrom("Hello from native!");

    Linker linker = Linker.nativeLinker();

    MethodHandle strlen = linker.downcallHandle(
        linker.defaultLookup().find("strlen").orElseThrow(),
        FunctionDescriptor.of(ValueLayout.JAVA_LONG, ValueLayout.ADDRESS)
    );

    long len = (long) strlen.invoke(str); // 17
```



```
} // arena.close() -> native memory freed automatically

// Record Patterns (Java 21) – deconstruct records in patterns
record Pair<A,B>(A first, B second) {}

Object obj = new Pair<>("hello", 42);

if (obj instanceof Pair(String s, Integer i)) {
    System.out.println(s + " " + i); // hello 42
}
```

*Foreign Function & Memory API and Record Patterns*

### 8.3 Summary

GraalVM Native Image enables instant-startup Java for serverless and containers, at the cost of dynamic runtime behaviour. Spring Boot 3 makes native first-class. Project Panama FF&M replaces JNI with safe native interop. Project Valhalla (Java 25+) will bring heap-flat value types for small immutable aggregates.

## 9 Modern Java Feature Timeline (Java 9–21)

| Version        | Key Features                                                                            | Type    |
|----------------|-----------------------------------------------------------------------------------------|---------|
| Java 9 (2017)  | JPMS modules, JShell, Collection factories, Stream.takeWhile/dropWhile, Optional.stream | Non-LTS |
| Java 10 (2018) | var (local-variable type inference), App Class-Data Sharing                             | Non-LTS |
| Java 11 (2018) | HTTP Client (standard), String.strip/isBlank/lines, var in lambda                       | LTS     |
| Java 12 (2019) | Switch expressions (preview), Shenandoah GC                                             | Non-LTS |
| Java 13 (2019) | Text Blocks (preview)                                                                   | Non-LTS |
| Java 14 (2020) | Records (preview), Pattern instanceof (preview), Helpful NPEs                           | Non-LTS |
| Java 15 (2020) | Text Blocks (standard), Sealed Classes (preview)                                        | Non-LTS |
| Java 16 (2021) | Records (standard), Pattern instanceof (standard), Stream.toList()                      | Non-LTS |
| Java 17 (2021) | Sealed Classes (standard), Enhanced PRNG, Remove SecurityManager                        | LTS     |
| Java 18 (2022) | UTF-8 by default, Simple Web Server                                                     | Non-LTS |
| Java 19 (2022) | Virtual Threads (preview), Structured Concurrency (incubator)                           | Non-LTS |
| Java 20 (2023) | Scoped Values (incubator), Record Patterns (preview)                                    | Non-LTS |
| Java 21 (2023) | Virtual Threads, Sequenced Collections, Pattern Switch, Record Patterns, Scoped Values  | LTS     |

## 10 Interview Q&A — Standard Questions (Q1–Q26)

### Q1. What problem does JPMS solve?

JPMS solves classpath hell, encapsulation leaks, and unreliable configuration. Before modules, all JARs were on the flat classpath — any class could access any other. Modules enforce strong encapsulation (only exported packages accessible), reliable configuration (missing/duplicate modules detected at startup), and allow the JDK to be modularised and stripped down.

### Q2. Difference between exports and opens in module-info.java?

exports makes a package accessible at compile time and runtime — but NOT for deep reflection. opens allows deep reflection (private field access via `java.lang.reflect`) at runtime without making the package importable at compile time. Spring and Hibernate rely on opens to inject fields and proxy classes.

### Q3. What does var do in Java 10 and what are its limitations?

var enables local-variable type inference. The compiler infers the static type from the right-hand side. Limitations: only for local variables (not fields, method params, or return types), initialiser is mandatory, cannot be null (type can't be inferred from null), not dynamic — once inferred the type is fixed. var is a reserved type name, not a keyword.

### Q4. What new String methods were added in Java 11?

`strip()/stripLeading()/stripTrailing()` — Unicode-aware whitespace removal (`trim()` only strips ASCII `<= \u0020`). `isBlank()` — true if empty or only whitespace. `lines()` — returns `Stream<String>` of lines. `repeat(n)` — repeats string `n` times. Java 12 added `indent(n)` and `transform(f)`; Java 15 added `formatted()` (`String.format` shortcut).

### Q5. What is a Record in Java 16 and what does it auto-generate?

A record is an immutable data carrier. `record Point(double x, double y) {}` auto-generates: private final fields, canonical constructor, accessor methods `x()` and `y()` (NOT `getX()`), `equals()` comparing all components, `hashCode()` based on all components, `toString()` like `'Point[x=1.0, y=2.0]'`. Records are implicitly final and cannot extend other classes.

### Q6. What is a sealed class and why is it useful?

A sealed class declares which classes can extend it: `'sealed interface S permits A,B'`. Permitted subclasses must be final, sealed, or non-sealed. Benefits: exhaustive pattern matching (compiler knows all subtypes), domain modelling (algebraic data types), security (prevents unintended extension).

### Q7. What is a Text Block in Java 15 and how does it handle indentation?

A multi-line string literal delimited by triple quotes. The compiler strips incidental leading whitespace based on the closing delimiter position. `stripIndent()` and `re-indent()` are also available. Escape `\s` preserves a trailing space; `\s` suppresses a newline. At runtime a text block is just a `String` constant — zero performance difference.

### Q8. What are Virtual Threads in Java 21?

Lightweight JVM-managed threads (Project Loom). They mount on carrier platform threads when running and unmount during blocking (I/O, sleep, lock). Unlike platform threads (1:1 OS threads, ~1 MB stack), millions coexist with minimal memory. Same Thread API — no code changes for blocking code. Best for high-concurrency I/O-bound workloads.

**Q9. What are Sequenced Collections in Java 21?**

Three new interfaces: `SequencedCollection` (adds `getFirst/getLast/addFirst/addLast/reversed`), `SequencedSet`, and `SequencedMap` (adds `firstEntry/lastEntry/reversed`). Retrofitted onto `ArrayList`, `Deque`, `LinkedHashSet`, `LinkedHashMap`. Solves inconsistent first/last access across collection types.

**Q10. What is Pattern Matching for switch in Java 21?**

Type-and-structure dispatch: `switch(obj) { case String s -> ... case Integer i when i>0 -> ... case null -> ... }`. Features: type patterns with binding variables, guarded patterns using `when`, null handling in case labels, exhaustiveness checking for sealed types, dominance rules (specific before general).

**Q11. What are Collection factory methods added in Java 9?**

`List.of()`, `Set.of()`, `Map.of()` (up to 10 pairs), `Map.ofEntries(Map.entry(k,v),...)`. These are immutable (`UnsupportedOperationException` on mutation), null-hostile (`NullPointerException` on null). `Set.of` and `Map.of` keys must be distinct.

**Q12. What Stream methods were added in Java 9?**

`takeWhile(pred)` — takes while true, stops; `dropWhile(pred)` — drops while true, passes remaining; `iterate(seed,pred,next)` — finite iterate; `ofNullable(t)` — empty if null, singleton otherwise. Java 16 added `Stream.toList()` as shortcut for `collect(Collectors.toUnmodifiableList())`.

**Q13. What is Structured Concurrency in Java 21?**

Treats concurrent subtasks as a unit. `StructuredTaskScope.ShutdownOnFailure` cancels all if one fails; `ShutdownOnSuccess` cancels remaining if one succeeds. Prevents thread leaks; scope must be joined before accessing results. Simplifies error handling vs raw `CompletableFuture`.

**Q14. What are Scoped Values in Java 21?**

Immutable alternative to `ThreadLocal` for virtual threads. Bound for a bounded scope: `ScopedValue.where(K,v).run(() -> K.get())`. Unlike `ThreadLocal`: immutable within scope, auto-inherited by child virtual threads, no memory leak risk, lower overhead at scale.

**Q15. What helpful NullPointerException messages were added in Java 14?**

Helpful NPEs (default from Java 15+) describe which variable was null: 'Cannot invoke `String.length()` because `str` is null'. Enable explicitly in earlier JVMs with `-XX:+ShowCodeDetailsInExceptionMessages`.

**Q16. What is the difference between LTS and non-LTS Java releases?**

Java releases every 6 months. LTS (Long-Term Support) versions — Java 8, 11, 17, 21 — receive extended security patches for several years and are recommended for production. Non-LTS versions receive updates only until the next release (6 months). Java 25 (September 2025) is the next LTS.

**Q17. What is the Foreign Function & Memory API in Java 21?**

The FF&M API (Project Panama, preview in Java 21; standard in Java 22) allows Java to call native libraries and manage off-heap memory safely. Key classes: `Arena` (manages native memory lifetime), `MemorySegment` (contiguous memory region), `Linker` (bridges Java and native ABI), `MethodHandle` (native function reference). Replaces unsafe `sun.misc.Unsafe` and `JNI`.

**Q18. What is GraalVM Native Image?**

GraalVM compiles Java AOT into standalone native executables using Substrate VM. Benefits: ~10ms startup (no JVM warmup), low memory footprint. Limitations: no dynamic class loading, reflection must be declared in config

files, no JIT profiling. Used by Quarkus, Micronaut, and Spring Native for microservices and serverless.

#### Q19. What are Record Patterns in Java 21?

Deconstruct records in instanceof and switch: `if (obj instanceof Pair(String s, Integer i))`. The pattern matches if obj is a Pair AND first component is String AND second is Integer, binding s and i. Nested patterns are supported. Combined with sealed types this enables exhaustive, type-safe data processing.

#### Q20. What changes were made to Optional in Java 9?

Java 9 added three methods: `ifPresentOrElse(consumer, runnable)` — action if present, else run empty-action; `or(supplier)` — returns another `Optional<T>` from supplier if empty (vs `orElseGet` which returns T); `stream()` — Stream of one element if present, empty stream otherwise. Useful for flatMapping: `optionals.stream().flatMap(Optional::stream)`.

#### Q21. What is Switch Expressions (Java 14) vs traditional switch?

Switch expressions: (1) return a value — usable in assignments; (2) arrow syntax `->` eliminates fall-through; (3) yield returns a value from a block case; (4) exhaustiveness enforced by compiler when used as expression. `'var result = switch(day) { case MON -> 1; case WED -> { yield compute(); } default -> 0; }'`

#### Q22. What is Stream.toList() added in Java 16?

`Stream.toList()` is a terminal operation that collects stream elements into an unmodifiable List. It is shorthand for `collect(Collectors.toUnmodifiableList())`. The returned list preserves encounter order and throws `UnsupportedOperationException` on mutation. Unlike `Collectors.toList()` which returns a mutable list.

#### Q23. What improvements were made to instanceof pattern matching in Java 16?

Binding variable is in scope only when condition is true: `if (obj instanceof String s) { s.length(); }`. Negation: `if (!(obj instanceof String s)) return;` — s is in scope after the early return. Guards: `obj instanceof String s && s.length() > 5`. Standard in Java 16 (was preview in Java 14).

#### Q24. What is the HTTP Client API in Java 11?

`java.net.http.HttpClient` supports HTTP/1.1 and HTTP/2, WebSockets, and both sync and async requests. Key classes: `HttpClient` (builder with version, timeout, authenticator, proxy, SSL), `HttpRequest` (URI, method, headers, body publisher), `HttpResponse` (status, headers, body handler). Async: `client.sendAsync()` returns `CompletableFuture`. Replaces `URLConnection`.

#### Q25. What is App Class-Data Sharing (CDS) in Java 10?

CDS serializes class metadata (class hierarchy, bytecodes, string pool) to a shared archive (.jsa file). Subsequent JVM startups memory-map the archive, reducing startup time 10-50% and RSS by 100-300 MB. Java 13 added Dynamic CDS (auto archive on exit). Java 17 made the default CDS archive always-on. Useful for microservices with short-lived JVM instances.

#### Q26. What are key changes to Process API in Java 9?

Java 9 added `ProcessHandle`: provides PID, info (command, arguments, start time, CPU time), `descendants()`, `parent()`, and `onExit()` (`CompletableFuture`). `ProcessBuilder.startPipeline()` chains processes like shell pipes. `ProcessHandle.current().info().command()` gives the JVM executable path. Useful for monitoring and building process pipelines from Java.

## 10b Interview Q&A — Advanced Questions (Q27–Q52)

### Q27. [ADV] How does JVM implement Virtual Thread mounting/unmounting under the hood?

Virtual threads run on carrier (platform) threads from a ForkJoinPool. When a virtual thread blocks (I/O, `Object.wait`, `Lock.lock`), the JVM performs a continuation yield: the virtual thread's stack is captured as a Continuation object on the heap, freeing the carrier. When the blocking op completes, the virtual thread is rescheduled on any available carrier and the continuation resumed. `synchronized` pins the carrier (fixed in Java 24+). Use `ReentrantLock` to avoid pinning.

### Q28. [ADV] Explain the JPMS split package problem.

A split package occurs when the same package name appears in two different modules. JPMS forbids this — the module system cannot determine which module's class to load. Common culprit: `java.xml.ws` split between the JDK and a third-party JAR. Solutions: shade (repackage) conflicting deps, use unnamed module (classpath), `--patch-module`. Split packages were allowed on the classpath but are a hard error in the module system.

### Q29. [ADV] How does the JVM's Continuation mechanism enable virtual threads?

Java 21 virtual threads are built on Continuations (internal `java.lang.Continuation`). A Continuation captures the entire call stack, allows it to yield (suspend), and later resume from the same point. Virtual threads wrap a Continuation — when blocked, `yield()` saves the stack to heap; `run()` resumes it. This is cooperative (not preemptive) scheduling at the JVM level. The ForkJoinPool scheduler decides when to mount/unmount.

### Q30. [ADV] What is the performance implication of using var vs explicit types?

At the bytecode level, `var` has zero performance impact — the compiler infers the type at compile time and emits identical bytecode to an explicit type declaration. Potential pitfall: `var list = new ArrayList<>()` infers `ArrayList` (raw-ish) not `List`, exposing implementation type. Always use `var` with a readable RHS so the inferred type is obvious.

### Q31. [ADV] How does pattern matching switch handle exhaustiveness with sealed types?

When switching on a sealed type, the compiler enumerates all permitted subtypes and verifies every case is covered. If a subtype is missing, compile error occurs (no default needed). With guards (when), exhaustiveness is conservative — a guarded case alone doesn't satisfy it (guard might be false). Adding a new permitted subtype breaks all exhaustive switches — by design, forcing callers to handle new cases.

### Q32. [ADV] What is pinning in virtual threads and how do you avoid it?

A virtual thread is pinned to its carrier when inside a `synchronized` block/method or calling native code. Pinning means the carrier cannot unmount, eliminating scalability. Diagnose: `-Djdk.tracePinnedThreads=full` or JFR `VirtualThreadPinned` event. Fix: replace `synchronized` with `ReentrantLock` or other `java.util.concurrent` locks, which support virtual thread unmounting during blocking.

### Q33. [ADV] How does the Foreign Function & Memory API compare to JNI?

JNI requires C header files, native stub compilation, and is error-prone. FF&M API: (1) pure Java — no C code for most native calls; (2) Arena-managed memory — automatic cleanup; (3) MethodHandle-based — type-safe and JIT-optimizable; (4) VarHandle for struct field access. Compared to JNI, FF&M is safer (bounds-checked), more performant (no GC pinning), and easier to use.

### Q34. [ADV] What is the difference between ShutdownOnFailure and ShutdownOnSuccess?

**ShutdownOnFailure:** when any child task fails, all remaining tasks are cancelled. `scope.join().throwIfFailed()` re-throws the first failure. Use when all results are needed (fail fast if any fails). **ShutdownOnSuccess:** when any child task succeeds, all remaining tasks are cancelled. `scope.result()` returns the first successful result. Use for racing alternatives (try two cache servers, use whichever responds first).

### Q35. [ADV] Explain Class-Data Sharing (CDS) and how it improves startup time.

CDS serializes class metadata (class hierarchy, method bytecodes, string pool) into a memory-mapped archive. On subsequent JVM startups, the archive is mmap'd — the OS can share physical pages across multiple JVM processes (CoW). This skips class parsing and verification, reducing startup 10-50% and RSS by 100-300 MB. Java 19+ added Dynamic CDS (auto archiving on exit). Combined with AOT (GraalVM) for maximum startup reduction.

### Q36. [ADV] How does type inference for var interact with generic types and wildcards?

`var` infers the most specific type from the RHS. `'var list = new ArrayList<>()'` infers `ArrayList<Object>` (erased), not `List` or `ArrayList<String>`. `'var nums = List.of(1,2,3)'` infers `List<Integer>`. Wildcards: `'var x = getWildcard()'` where return is `List<?>` infers `List<?>` — `x.add()` is then forbidden. Intersection types from lambdas: if `var` captures a lambda, the type is the functional interface.

### Q37. [ADV] What are the memory model implications of virtual thread handoff between carriers?

When a virtual thread unmounts from carrier A and remounts on carrier B, the JVM inserts happens-before edges at mount/unmount points — the virtual thread sees all writes made before unmounting, and all writes on the new carrier before mounting are visible. This is specified in the Java Memory Model extension for virtual threads. A virtual thread has a consistent view of its own state regardless of which carrier it runs on.

### Q38. [ADV] How does GraalVM SubstrateVM handle reflection in native images?

SubstrateVM performs closed-world analysis — all code reachable at runtime must be known at build time. Dynamic reflection, proxy generation, and serialization require configuration. GraalVM agent: run app with `'-agentlib:native-image-agent=config-output-dir=META-INF/native-image'` to auto-generate `reflect-config.json`, `proxy-config.json`, `serialization-config.json`. Frameworks like Quarkus/Micronaut generate these at build time via annotation processing.

### Q39. [ADV] What is the relationship between Records and Java serialization?

Records are serializable if they implement `Serializable`. The serialization mechanism for records uses the canonical constructor (not `readObject/readResolve`). This means: (1) serialization cannot bypass validation in the compact constructor; (2) all components must be serializable; (3) `serialVersionUID` is optional. This fixes a long-standing vulnerability — records cannot have inconsistent state injected via deserialization.

### Q40. [ADV] How does the JVM support multiple concurrent versions of modules?

JPMS does not natively support multiple versions of the same module in one layer. However, module layers (`ModuleLayer`) allow loading a module in a child layer with a different version — each layer has its own classloader. OSGi solves this with its own class-loading framework. Maven shading (repackaging) is the pragmatic solution for dependency conflicts. The unnamed module (classpath) and automatic modules provide backward compatibility for non-modularised JARs.

### Q41. [ADV] What are the performance characteristics of Text Blocks vs String concatenation?

Text Blocks are a compile-time feature — compiled to regular String literals with processed indentation and escape sequences. At runtime, a text block is just a String constant in the constant pool — identical performance to a string literal. `String.formatted()` (called on a text block) is equivalent to `String.format()` and involves runtime formatting cost. Indentation stripping happens at compile time, not runtime.



**Q42. [ADV] Explain how `Stream.takeWhile` differs from `filter` when used with ordered streams.**

`takeWhile` is a short-circuiting stateful intermediate operation: takes elements sequentially until the predicate fails, then stops — subsequent elements matching the predicate are NOT included. `filter` evaluates the predicate on every element independently. `Stream.of(2,4,6,5,8).takeWhile(n->n%2==0)` gives `[2,4,6]` (stops at 5). `filter` gives `[2,4,6,8]` (evaluates all). For parallel streams, `takeWhile` is more expensive (must respect encounter order).

**Q43. [ADV] What is a Hidden Class in Java 15 and what is it used for?**

Hidden classes cannot be used directly by the bytecode of other classes and are intended for frameworks that generate classes at runtime. Created via `Lookup.defineHiddenClass()`. Key properties: not discoverable by name in `ClassLoader`; can be unloaded independently of their defining classloader; shorter lifecycle — GC'd when no references remain. Used by lambda implementation (replaces anonymous classes generated by `LambdaMetafactory`), byte-buddy, ASM for dynamic proxy generation.

**Q44. [ADV] How does JEP 425 (Virtual Threads) interact with `ThreadLocal`?**

`ThreadLocal` works with virtual threads — each has its own `ThreadLocal` storage. Problem: with virtual-thread-per-task, each task gets a fresh virtual thread — `ThreadLocal` is isolated per task (better than platform thread pools). However, `ThreadLocal` still allocates a `Map` per virtual thread, expensive at millions of threads. The preferred solution is `Scoped Values` — immutable, inherited, lower memory overhead.

**Q45. [ADV] What is the significance of exhaustiveness in sealed-type pattern switches for API design?**

Sealed types + exhaustive switches create a visitor pattern built into the language. When a library adds a new permitted subtype, all exhaustive switches in client code break at compile time — forcing clients to handle the new case. This makes APIs evolvable with compile-time safety rather than runtime surprises. Compare to interface + `instanceof` chains which silently ignore new subtypes. For internal APIs, this is a major advantage over non-sealed hierarchies.

**Q46. [ADV] How does the JVM optimise records at the JIT level?**

Records benefit from: (1) Scalar replacement — small records on a hot path may be decomposed into fields and stack-allocated; (2) Inlining — accessor methods (`x()`, `y()`) are trivial and always inlined; (3) Escape analysis — if a record does not escape a method, it may never be heap-allocated. Project Valhalla (future) will introduce value types — records without object identity — enabling even more aggressive flattening in arrays and fields.

**Q47. [ADV] What makes `Optional.or()` different from `Optional.orElseGet()` in Java 9?**

`orElseGet(Supplier<T>)` returns a plain `T` value if the `Optional` is empty. `or(Supplier<Optional<T>>)` returns another `Optional<T>` if empty, preserving the wrapper. This enables chaining: `opt.or() -> cache.get(key)).or() -> db.find(key)).orElseThrow()`. Without `or()`, nested `ifPresent` checks or `flatMap` tricks were needed. `or()` preserves the `Optional` wrapper allowing further `Optional` transformations.

**Q48. [ADV] Explain the Memory Segment and Arena model in the Foreign Memory API.**

A `MemorySegment` is a contiguous region of memory (on-heap or off-heap) with a spatial bound (size) and a temporal bound (lifetime controlled by `Arena`). `Arena` types: `confined` (single-thread, deterministic close), `shared` (multi-thread), `auto` (GC-managed, non-deterministic), `global` (never closed). Accessing a `MemorySegment` after its `Arena` is closed throws `IllegalStateException`. `VarHandle` or `MemoryLayout` provides typed access to struct fields. All accesses are bounds-checked — no segfaults.

**Q49. [ADV] How does JShell implement incremental compilation and state?**



JShell maintains a state machine — each snippet (expression, statement, declaration, import) is wrapped in a synthetic class and compiled incrementally. When a snippet redefines a variable or method, JShell replaces the previous snippet and recompiles dependents. Snippets execute via class loading into a remote JVM. State (variables, methods, imports) persists across snippets. The JShell API (javax.tools.JavaCompiler-based) allows embedding REPL functionality in tools.

#### **Q50. [ADV] What are the implications of removing SecurityManager in Java 17?**

SecurityManager was deprecated in Java 17 (JEP 411) and removed in Java 24. It provided policy-based sandboxing but was rarely configured correctly, had performance overhead on every privileged operation, and didn't work with modern container-based isolation. Replacements: OS-level sandboxing (seccomp, AppArmor, containers), JPMS strong encapsulation, Java agent-based monitoring. Libraries relying on `AccessController.doPrivileged()` need migration.

#### **Q51. [ADV] How does dynamic CDS (Class Data Sharing) work with Spring Boot?**

Spring Boot 3.x supports CDS natively. Process: (1) `'java -Dspring.context.exit=onRefresh -jar app.jar'` performs a training run and dumps the archive; (2) subsequent runs: `'java -XX:SharedArchiveFile=app.jsa -jar app.jar'` use the archive. The CDS archive includes Spring's class metadata, parsed bytecodes, and string pools. Combined with checkpoint/restore (Project CRaC), Spring Boot apps can achieve sub-100ms startup times.

#### **Q52. [ADV] What is the difference between value-based classes and value types (Project Valhalla)?**

Value-based classes (e.g., `Optional`, `LocalDate`, Integer wrappers) are existing classes where identity operations (`==`, `synchronize`, `identityHashCode`) are discouraged — flagged with `@jdk.internal.ValueBased`. Value types (Project Valhalla, Java 25+): primitive-like objects flattened in memory with no object header, no identity. Declared with `'value class'`. Allow JVM to store them inline in arrays and fields — eliminating indirection and GC pressure for small immutable aggregates like `Point`, `Range`, `Complex`.

## 11 Coding Exercises

### Exercise 1: Sealed Shape Hierarchy with Exhaustive Switch

Define a sealed interface `Shape` with records `Circle`, `Rectangle`, `Triangle`. Write `area(Shape)` using an exhaustive pattern switch. Adding a new `Shape` subtype must cause a compile error.

```
public sealed interface Shape permits Circle, Rectangle, Triangle {}

public record Circle(double radius) implements Shape {}

public record Rectangle(double w, double h) implements Shape {}

public record Triangle(double base, double height) implements Shape {}

public class ShapeCalculator {

    public static double area(Shape s) {

        return switch (s) {

            case Circle c -> Math.PI * c.radius() * c.radius();

            case Rectangle r -> r.w() * r.h();

            case Triangle t -> 0.5 * t.base() * t.height();

            // No default: compiler enforces exhaustiveness

        };

    }

    public static void main(String[] args) {

        System.out.println(area(new Circle(5))); // 78.53...

        System.out.println(area(new Rectangle(4,6))); // 24.0

        System.out.println(area(new Triangle(3,8))); // 12.0

    }

}
```

*Exercise 1 — Sealed + Pattern Switch (no extra imports)*

### Exercise 2: Concurrent Fetch with Structured Concurrency

Fetch two URLs concurrently using virtual threads and `StructuredTaskScope.ShutdownOnFailure`. Combine results. Handle failure with `throwIfFailed`.

```
import java.net.URI;

import java.net.http.HttpClient;

import java.net.http.HttpRequest;
```

```
import java.net.http.HttpResponse;
import java.util.concurrent.StructuredTaskScope;
import java.util.concurrent.StructuredTaskScope.Subtask;

record FetchResult(String url, int status, long ms) {}

public class ConcurrentFetcher {
    private final HttpClient http = HttpClient.newHttpClient();
    record Report(FetchResult r1, FetchResult r2) {}

    public Report fetchBoth(String url1, String url2) throws Exception {
        try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
            var f1 = scope.fork(() -> fetch(url1));
            var f2 = scope.fork(() -> fetch(url2));
            scope.join().throwIfFailed();
            return new Report(f1.get(), f2.get());
        }
    }

    private FetchResult fetch(String url) throws Exception {
        long t = System.currentTimeMillis();
        var resp = http.send(
            HttpRequest.newBuilder().uri(URI.create(url)).GET().build(),
            HttpResponse.BodyHandlers.discarding());
        return new FetchResult(url, resp.statusCode(),
            System.currentTimeMillis() - t);
    }
}
```

*Exercise 2 — Virtual Threads + Structured Concurrency*

### Exercise 3: Text Block JSON Builder with Records

Build a JSON array from a list of Person records using text blocks and formatted().

```
import java.util.List;
import java.util.stream.Collectors;

record Person(String name, int age, String email) {}

public class JsonBuilder {
    public static String toJson(List<Person> people) {
        var entries = people.stream()
```

```
.map(p -> ""
{
    "name": "%s",
    "age": %d,
    "email": "%s"
}""formatted(p.name(), p.age(), p.email()))
.collect(Collectors.joining(",\n"));
return ""
[
%S
]""formatted(entries);
}
}
```

*Exercise 3 — Text Blocks + Records*

## Exercise 4: Sliding Window Average with Sequenced Collections

Use `ArrayDeque` (`SequencedCollection`) to maintain a sliding window of `N` integers, computing the running average.

```
import java.util.ArrayDeque;

public class SlidingWindowAverage {
    private final ArrayDeque<Integer> window;
    private final int capacity;
    private long sum = 0;

    public SlidingWindowAverage(int n) {
        capacity = n;
        window = new ArrayDeque<>(n);
    }

    public double add(int v) {
        if (window.size() == capacity)
            sum -= window.removeFirst(); // SequencedCollection API
        window.addLast(v); // SequencedCollection API
        sum += v;
        return (double) sum / window.size();
    }

    public static void main(String[] args) {
        var avg = new SlidingWindowAverage(3);
```

```
System.out.println(avg.add(1)); // 1.0
System.out.println(avg.add(2)); // 1.5
System.out.println(avg.add(3)); // 2.0
System.out.println(avg.add(7)); // 4.0 (drops 1)
}
}
```

*Exercise 4 — Sequenced Collections*

## Exercise 5: Module-aware ServiceLoader with JPMS

Define a GreeterService SPI, two implementations, and wire them via module-info.java using uses/provides directives and ServiceLoader.

```
import com.example.api.GreeterService;
import java.util.ServiceLoader;

// --- module-info.java (api module) ---
module com.example.api {
    exports com.example.api;
    uses com.example.api.GreeterService;
}

// --- GreeterService.java ---
package com.example.api;

public interface GreeterService { String greet(String name); }

// --- module-info.java (impl module) ---
module com.example.impl {
    requires com.example.api;
    provides com.example.api.GreeterService
        with com.example.impl.EnglishGreeter,
            com.example.impl.FrenchGreeter;
}

// --- Main.java ---
ServiceLoader<GreeterService> loader =
    ServiceLoader.load(GreeterService.class);
loader.forEach(g -> System.out.println(g.greet("World")));
```

*Exercise 5 — JPMS + ServiceLoader*

## 12 Debug & Fix Scenarios

### Debug 1: Virtual Thread Pinning Causing Starvation

```
import java.util.Queue;
import java.util.LinkedList;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

// BUG: synchronized pins carrier thread
public synchronized Connection borrow() throws InterruptedException {
    while (pool.isEmpty()) Thread.sleep(10); // carrier PINNED!
    return pool.poll();
}

// FIX: ReentrantLock allows carrier to unmount during await()
private final ReentrantLock lock = new ReentrantLock();
private final Condition notEmpty = lock.newCondition();

public Connection borrow() throws InterruptedException {
    lock.lock();
    try {
        while (pool.isEmpty()) notEmpty.await(); // yields carrier
        return pool.poll();
    } finally {
        lock.unlock();
    }
}
```

*Debug 1 — Virtual Thread Pinning Fix*

### Debug 2: Record Field Mutation via Reflection

```
import java.lang.reflect.Field;

// BUG: record components are private final – cannot be mutated
record Temperature(double celsius) {
    Temperature { if (celsius < -273.15) throw new IllegalArgumentException(); }
}

Field f = Temperature.class.getDeclaredField("celsius");
f.setAccessible(true);
```

```
f.setDouble(new Temperature(25.0), -999); // throws IllegalAccessException

// CORRECT: records are immutable – create a new instance
Temperature original = new Temperature(25.0);
Temperature adjusted = new Temperature(original.celsius() + 5); // 30.0
```

*Debug 2 — Record Immutability*

## Debug 3: JPMS Missing opens for Spring

```
// BUG: InaccessibleObjectException – Spring cannot inject private fields

// module-info.java – BROKEN

module com.app {

    requires spring.core;

    exports com.app.api;

    // MISSING: no opens -> InaccessibleObjectException at runtime

}

// FIXED

module com.app {

    requires spring.core;

    exports com.app.api;

    opens com.app.service to spring.core; // allow deep reflection

}
```

*Debug 3 — JPMS opens Fix (module-info.java — no imports)*

## Debug 4: Text Block Indentation Pitfall

```
// BUG: closing "" at column 0 – all 8 leading spaces RETAINED

String sql = ""

SELECT *

FROM users

""; // closing at col 0 -> " SELECT *\n FROM users\n"

// FIX: align closing "" with content – 8 spaces stripped

String sql = ""

SELECT *

FROM users
```

```
"""; // -> "SELECT *\nFROM users\n"
```

*Debug 4 — Text Block Indentation (java.lang.String — no imports needed)*



## 13 Quick Reference Cheat Sheet

| Feature                       | Key Syntax / Detail                                                                                       | Version |
|-------------------------------|-----------------------------------------------------------------------------------------------------------|---------|
| JPMS Module                   | <code>module x { requires y; exports p; opens p to m; }</code>                                            | Java 9  |
| Collection Factories          | <code>List.of()</code> , <code>Set.of()</code> , <code>Map.of()</code> — immutable, null-hostile          | Java 9  |
| <code>Stream.takeWhile</code> | <code>.takeWhile(pred)</code> — stops at first false (unlike filter)                                      | Java 9  |
| <code>Optional.or()</code>    | <code>opt.or()</code> -> <code>Optional.of(fallback)</code> — chain optionals                             | Java 9  |
| <code>var</code> keyword      | Local only; compile-time inference; not dynamic typing                                                    | Java 10 |
| <code>String.strip()</code>   | Unicode-aware trim; <code>isBlank()</code> , <code>lines()</code> , <code>repeat(n)</code>                | Java 11 |
| HTTP Client                   | <code>HttpClient.newBuilder().version(HTTP_2).build()</code>                                              | Java 11 |
| Switch Expression             | <code>switch(x) { case A -&gt; val; case B -&gt; { yield v; } }</code>                                    | Java 14 |
| Helpful NPEs                  | <code>-XX:+ShowCodeDetailsInExceptionMessages</code> (on by default Java 15+)                             | Java 14 |
| Text Blocks                   | <code>""" ... """</code> — indent stripped at compile time                                                | Java 15 |
| Records                       | <code>record P(int x, int y) {}</code> — final, immutable, auto-methods                                   | Java 16 |
| instanceof patterns           | <code>if (obj instanceof String s &amp;&amp; s.length() &gt; 5)</code>                                    | Java 16 |
| <code>Stream.toList()</code>  | Shortcut for <code>collect(Collectors.toUnmodifiableList())</code>                                        | Java 16 |
| Sealed Classes                | sealed interface <code>S</code> permits <code>A,B</code> ; subclass: <code>final/sealed/non-sealed</code> | Java 17 |
| Virtual Threads               | <code>Thread.ofVirtual().start(...)</code> ;<br><code>Executors.newVirtualThreadPerTaskExecutor()</code>  | Java 21 |
| Pattern Switch                | <code>switch(s) { case Circle c when c.r()&gt;0 -&gt; ...; case null -&gt; ...; }</code>                  | Java 21 |
| SequencedCollection           | <code>getFirst/getLast/addFirst/addLast/reversed()</code> on <code>List/Deque</code>                      | Java 21 |
| Scoped Values                 | <code>ScopedValue.where(K,v).run(() -&gt; K.get())</code> — immutable <code>ThreadLocal</code> alt        | Java 21 |
| Record Patterns               | <code>if (obj instanceof Pair(String s, Integer i)) { ... }</code>                                        | Java 21 |
| FF&M API                      | <code>Arena.ofConfined()</code> ; <code>MemorySegment</code> ;<br><code>Linker.nativeLinker()</code>      | Java 21 |